

ANALISI DOCUMENTAZIONE

“CLEANING 1-2-3(4)”

Legenda:

Cleaning 1

1. Import dei file da analizzare più i file di supporto
2. Operazione per popolare il support file per i file considerati →
Generico

```
support_file_path = 'ADNI_variables_statistics'  
  
support_file = pd.read_excel(support_file_path+'.xlsx')
```

Il codice definisce il percorso del **support file** (ADNI_variables_statistics.xlsx) e ne carica il contenuto in un DataFrame Pandas mediante `pd.read_excel()`. Il DataFrame `support_file` contiene i metadati e le informazioni descrittive delle variabili utilizzate nella pipeline di preprocessing e armonizzazione dei dati.

- Questo script ha lo scopo di aggiornare automaticamente il **support file** contenente i metadati delle variabili presenti nei dataset ADNI. Per ogni file contenuto nella collezione `zip_files`, viene recuperato il relativo DataFrame e viene creato un oggetto `InfoSupportFile`, che mette in relazione il dataset corrente con il support file.

```
for file_name in list(zip_files.keys()):  
  
    df = zip_files[file_name]  
  
    infoSupportFile = InfoSupportFile(  
        support_file,  
        df,  
        file_name  
    )
```

- Successivamente, il support file viene filtrato per eliminare le variabili che non sono effettivamente presenti nel dataset analizzato. Da questa operazione viene inoltre

ricavato il file_code, ovvero il codice identificativo del dataset.

- `filter_variable` (#generico) → ha lo scopo di ridurre il numero di colonne del DataFrame mantenendo esclusivamente le variabili previste nel support file per uno specifico dataset.

```
support_file, file_code = infoSupportFile.filter_variables()

print(file_code)
```

- Se il file_code non è presente nel support file, il dataset viene ignorato e l'elaborazione passa al file successivo.

```
if file_code not in list(support_file['file_code']):

    print('not found in excel')

    continue
```

- Per i dataset validi, il codice cerca di individuare automaticamente la variabile che identifica la coorte di appartenenza del soggetto (ad esempio ADNI1, ADNI2, ADNIGO, ADNI3 o ADNI4). Se questa informazione non è ancora documentata nel support file, viene aggiunta automaticamente.
 - `find_population_variable` (#generico) → ha lo scopo di identificare automaticamente la variabile che indica la coorte ADNI di appartenenza dei soggetti (ad esempio ADNI1, ADNIGO, ADNI2, ADNI3 o ADNI4) e di aggiornare il support file con questa informazione.

```
pop, support_file = infoSupportFile.find_population_variable()
```

- Successivamente, il programma analizza tutte le colonne del DataFrame.

```
for key in df.keys():
```

- Per ogni variabile presente nel support file viene verificata la sua esistenza e, in caso positivo, viene avviata l'analisi descrittiva della variabile.
 - `get_variable_info` (#generico) → Funzione di profiling e valutazione della qualità delle variabili, che aggiorna il file di supporto con statistiche, metadati e una decisione automatica sulla conservazione (keep) o eliminazione (drop) della variabile

```
if key in support_file[

    support_file['file_code'] = file_code

]['variable_code'].values:

    infoSupportFile.get_variable_info(key)
```

Durante questa fase vengono calcolate diverse informazioni descrittive e di qualità dei dati, tra cui:

- il tipo della variabile (numerica, categorica, stringa, ecc.);
 - l'intervallo dei valori per le variabili numeriche;
 - le classi presenti per le variabili categoriche (stringa e numerico);
 - il numero di valori validi e mancanti;
 - le popolazioni ADNI in cui la variabile è assente;
 - una valutazione automatica sulla conservazione o eliminazione della variabile (keep o drop) in base alla percentuale di dati disponibili.
- Le informazioni vengono ottenute tramite:

```
infoSupportFile.get_variable_info(key)
```

- che a sua volta richiama funzioni come:

```
n_tot, n_valid, n_missing, _, pop_missing = self.check_missing_values()
tipo, intervallo, classes = self.check_type_range_variables()
```

- e aggiorna il support file con i metadati raccolti:

```
self.support_file['type_variable'][index] = str(tipo)
self.support_file['classes'][index] = ', '.join(map(str, classes))
self.support_file['range'][index] = ', '.join(map(str, intervallo))
self.support_file['valid_values'][index] = int(n_valid)
self.support_file['missing_values'][index] = int(n_missing)
self.support_file['missing_pop'][index] = ', '.join(map(str, pop_missing))
```

- Inoltre, la funzione valuta automaticamente la qualità della variabile e decide se mantenerla o suggerirne l'eliminazione:

```
if n_tot > 0:
    if n_valid / n_tot <= 0.65:
        self.support_file['del'][index] = 'drop'
    else:
        self.support_file['del'][index] = 'keep'
```

Tutte queste informazioni vengono salvate direttamente nelle corrispondenti righe del support file, arricchendolo progressivamente con metadati utili per le successive fasi di armonizzazione, controllo qualità e preprocessing.

- Al termine dell'elaborazione di tutti i dataset, il support file aggiornato viene salvato

su disco.

```
save_df(  
    df_to_save=support_file,  
    output_path=support_file_path  
)
```

In questo modo si ottiene un catalogo completo e continuamente aggiornato delle variabili disponibili, delle loro caratteristiche e del loro livello di qualità all'interno dell'intero ecosistema di dati ADNI.

3. Pulizia file per file

Questo blocco di codice si occupa di **caricare il support file originale e creare oppure aggiornare una versione pulita e consolidata del file dei metadati**.

- Per prima cosa viene definito il nome del support file contenente le statistiche e le informazioni sulle variabili:

```
support_file_path = 'ADNI_variables_statistics'  
  
support_file = pd.read_excel(support_file_path+'.xlsx')
```

Attraverso `pd.read_excel()` il file Excel viene caricato in un DataFrame Pandas (`support_file`), che verrà utilizzato come base per la generazione della nuova versione dei metadati.

- Successivamente viene definito il nome del nuovo support file:

```
new_support_file_name = 'ADNI_variables_cleaned1'  
..
```

- Il codice verifica quindi se esiste già una versione del file con questo nome:

```
if os.path.isfile(new_support_file_name+'.xlsx'):  
..
```

- Se il file è già presente, significa che esiste una versione precedentemente elaborata del support file. In questo caso viene eseguito un aggiornamento tramite:
 - `update_new_support_file(#generico)` → Aggiorna il file di supporto con i nuovi file

```
update_new_support_file(  
    support_file,
```

```

new_support_file_name,

processed_file=file_codes

)

```

La funzione aggiorna il file esistente integrando le nuove informazioni ottenute dai dataset recentemente processati e mantenendo traccia dei file già elaborati.

- Se invece il file non esiste ancora, viene creata una nuova versione del support file mediante:
 - `create_new_support_file(#generico)` → crea una copia di ``self.support_file``, aggiunge la colonna `'new_variable_code'`, salva il nuovo DataFrame in un file Excel e lo restituisce. Il nuovo file viene salvato nella stessa directory dell'originale, con `'new_'` aggiunto all'inizio del nome del file.

```

create_new_support_file(

support_file,

support_file_path,

new_name=new_support_file_name

)

```

In questo modo viene generato un nuovo file Excel contenente la struttura aggiornata dei metadati e tutte le informazioni raccolte durante il processo di analisi delle variabili.

4. File specific data cleaning

- ☐ Recuperare un dataset specifico dal Data Lake, **scaricarlo, estrarlo e prepararlo** per le successive operazioni di analisi e preprocessing
 - Per prima cosa viene definito il codice identificativo del dataset da recuperare:

```
file_code = 'UPENNBIOBK_ROCHE_ELECSYS' #specifico per ciascun file
```

- Successivamente viene effettuata una ricerca nel Data Lake utilizzando il client, filtrando i file che appartengono al livello (level) *raw* e che possiedono il `file_code` specificato.

```

search = client.query_files(

query={

'custom.level' : 'raw',

'custom.file_code': file_code

}

)

```

Il risultato della ricerca contiene le informazioni necessarie per localizzare il file desiderato all'interno del Data Lake.

- Una volta identificato il file, questo viene scaricato tramite:

```
zip_files = client.download_file(  
    search['object_name'],  
    extract_zip=True  
)
```

L'opzione `extract_zip=True` indica che, se il file è compresso, il contenuto viene automaticamente estratto e restituito sotto forma di dizionario, in cui le chiavi rappresentano i nomi dei file e i valori contengono i relativi DataFrame.

- Successivamente viene recuperato il nome del primo file estratto:

```
file_name = list(zip_files.keys())[0]
```

- e viene caricato il dataset associato:

```
dataset = zip_files[file_name]
```

- Infine viene creata una copia profonda del Dataframe:

```
df_new = dataset.copy(deep=True)
```

L'utilizzo di una copia profonda (`deep=True`) garantisce che tutte le modifiche effettuate nelle successive fasi di preprocessing non alterino il dataset originale scaricato dal Data Lake.

In sintesi:

Il codice ricerca nel Data Lake il dataset identificato dal `file_code` specificato, ne esegue il download e l'estrazione, carica il contenuto in un DataFrame Pandas e crea una copia indipendente del dataset che verrà utilizzata nelle successive fasi di elaborazione e armonizzazione dei dati.

- ☐ **#specifo** → Questo blocco si occupa di un'ulteriore fase di pulizia dei dati, focalizzata su valori mancanti/sconosciuti e sulla standardizzazione di un parametro di controllo qualità.
- Per prima cosa viene definita una lista di colonne considerate fondamentali, da verificare con particolare attenzione:

```
columns_must_be_verified = ['BRAIN', 'EICV', 'VENTRICLES', 'LHIPPOC', 'RHIPPOC']
```

Questi nomi corrispondono a misure volumetriche cerebrali tipiche delle analisi di

neuroimaging: BRAIN (volume cerebrale totale), EICV (Estimated Intracranial Volume, il volume intracranico stimato, spesso usato per normalizzare le altre misure), VENTRICLES (volume dei ventricoli cerebrali), e LHIPPOC/RHIPPOC (volume dell'ippocampo sinistro e destro, strutture particolarmente rilevanti ad esempio negli studi sull'Alzheimer, coerentemente con il riferimento ad ADNI visto in precedenza).

- Successivamente viene chiamato un metodo per sistemare i valori sconosciuti nel DataFrame `df_new`:

```
no_unknow_df = dataCleaner.replace_unknown_values(df_new)
```

Questa funzione probabilmente individua valori "segnaposto" che indicano dati mancanti o non validi (ad esempio stringhe come "unknown", "N/A", o codici numerici convenzionali) e li sostituisce con un valore standard riconosciuto da pandas come mancante (tipicamente NaN), in modo da poterli gestire correttamente nelle fasi successive.

- Il DataFrame risultante viene poi passato a un secondo metodo che elimina le righe prive di dati rilevanti:

```
no_none_df = dataCleaner.drop_if_all_none(no_unknow_df, columns_must_be_verified)
```

Questa funzione controlla, per ogni riga, i valori nelle colonne elencate in `columns_must_be_verified` (le misure cerebrali fondamentali definite sopra) e scarta le righe in cui **tutti** questi valori risultano mancanti/nulli — il nome stesso della funzione, `drop_if_all_none`, suggerisce che la riga viene eliminata solo se *nessuna* delle colonne chiave contiene un dato utile, mentre viene mantenuta se anche solo una di esse ha un valore valido.

- A questo punto viene applicata una trasformazione sulla colonna QCPASS, che rappresenta l'esito del controllo qualità

```
no_none_df = dataCleaner.convert_qcpass_values(no_none_df, col_name='QCPASS')
```

Come indicato dal commento (`# Convert QCPASS values from 1/0 to 'complete'/'partial'`), questa funzione converte i valori binari 1/0 della colonna QCPASS in etichette testuali più descrittive, 'complete' e 'partial', presumibilmente per rendere il dato più leggibile e interpretabile nelle fasi successive di analisi o negli export finali.

- Infine, l'ultima riga è commentata e quindi non viene eseguita:

```
#no_none_df = dataCleaner.segmentation_complete_filter(no_none_df,  
filter_col='QCPASS')
```

Questa riga, se attivata, richiamerebbe un metodo `segmentation_complete_filter` per filtrare il DataFrame mantenendo solo le righe con segmentazione "completa" secondo la colonna QCPASS appena convertita. Il fatto che sia commentata suggerisce che questo passaggio sia stato temporaneamente disattivato, forse perché si è deciso — almeno in questa fase della pipeline — di conservare anche i casi con controllo qualità

"parziale", rimandando l'eventuale filtro a un momento successivo dell'elaborazione.

In sintesi:

Questo frammento rende il dataset più pulito e coerente: uniforma i valori sconosciuti, elimina le righe prive di informazioni utili sulle misure cerebrali chiave, e trasforma l'indicatore di qualità in un formato più leggibile, lasciando però commentata (quindi non attiva) l'opzione di filtrare esplicitamente solo i casi con qualità "completa".

- ☐ Fase preliminare di **pulizia e filtraggio dei dati**, obiettivo di rimuovere valori non validi e osservazioni prive delle informazioni biologiche essenziali per le analisi successive.
- Per prima cosa viene applicata la funzione:

```
no_unknow_df = dataCleaner.replace_unknown_values(df_new)
```

- `replace_unknown_values(#generico)` → questa operazione sostituisce con valori mancanti (NaN) tutte le occorrenze di codifiche utilizzate per rappresentare dati assenti o sconosciuti, come ad esempio:

```
Unknown
unknown
-4
9999
9999.0
```

In questo modo i dati mancanti vengono standardizzati e possono essere gestiti correttamente nelle successive fasi di elaborazione.

- Quindi viene definita una lista di biomarcatori considerati indispensabili:

```
columns_must_be_verified = ['ABETA42', 'TAU', 'PTAU']
```

Le variabili indicate nella lista vanno definite file per file in base ai dati e scopo del file.

- Una volta uniformata la rappresentazione dei valori mancanti, viene applicato un ulteriore filtro:

```
no_none_df = dataCleaner.drop_if_all_none(
    no_unknow_df,
    columns_must_be_verified
)
```


- `drop_if_all_none(#generica)` → la funzione controlla le colonne:

```
['ABETA42', 'TAU', 'PTAU']
```

ed elimina tutte le righe in cui i biomarcatori della lista sopra definita risultano contemporaneamente assenti (NaN). Vengono quindi mantenute solamente le osservazioni che contengono almeno una variabile utile tra quelle considerate.

Questa fase consente di ridurre il rumore presente nel dataset e di conservare esclusivamente i record con informazioni rilevanti per le analisi dei biomarcatori.

In sintesi:

Il codice standardizza la rappresentazione dei dati mancanti sostituendo valori speciali con NaN e rimuove successivamente tutte le osservazioni prive dei biomarcatori fondamentali da definire per ciascun file, mantenendo soltanto le righe contenenti almeno un'informazione biologica utile.

- ☐ **Uniformare il formato delle date delle visite e ricostruire correttamente la sequenza temporale delle osservazioni di ciascun soggetto**, identificando eventuali visite duplicate e calcolando la posizione cronologica delle visite.
 - Per prima cosa, la colonna contenente la data della visita viene convertita in un formato standard tramite:

```
datefix_df = dataCleaner.to_date_format(  
  
    no_none_df,  
  
    ['EXAMDATE']  
  
)
```

La funzione `to_date_format()` trasforma i valori della colonna `EXAMDATE` nel formato `datetime.date`, gestendo eventuali formati eterogenei o valori non validi. Questa operazione garantisce che tutte le date siano rappresentate in modo coerente e possano essere utilizzate correttamente nelle elaborazioni successive.

- Successivamente viene applicata la funzione:

```
datefix_df = dataCleaner.find_exam_code(  
  
    datefix_df,  
  
    date_column='EXAMDATE',  
  
    viscode_reference='VISCODE2',  
  
    patient_id_column='RID',
```

```
essential_variables=columns_must_be_verified
```

```
)
```

Questa funzione analizza le visite di ciascun soggetto identificato dalla variabile RID e procede a una serie di controlli e operazioni:

- ordina cronologicamente le visite utilizzando la colonna EXAMDATE;
- individua eventuali visite duplicate con la stessa data;
- risolve i duplicati selezionando la riga più informativa sulla base dei biomarcatori contenuti in columns_must_be_verified (file specifica);
- utilizza la variabile di riferimento VISCODE2/VISCODE per aiutare la selezione della visita corretta nei casi ambigui;
- calcola una variabile temporale (VISIT_MONTH) che rappresenta il numero di mesi trascorsi dalla visita basale del soggetto.

Il risultato finale è un dataset temporalmente coerente, privo di visite duplicate non necessarie e organizzato secondo la corretta sequenza delle osservazioni longitudinali.

In sintesi:

Il codice standardizza il formato delle date delle visite e successivamente ricostruisce la sequenza cronologica delle osservazioni per ciascun soggetto, risolvendo eventuali duplicati e calcolando la distanza temporale dalla visita basale attraverso la variabile VISIT_MONTH.

- ☐ Completa la fase di armonizzazione del dataset **assegnando il metodo di misura utilizzato per i biomarcatori** e selezionando esclusivamente le variabili previste dal support file.
- Viene aggiunta una nuova colonna denominata METHOD, valorizzata con il nome della piattaforma analitica utilizzata per la misurazione dei biomarcatori(#specific #CSF #PET #PLASMA #VOLUME):

```
datefix_df['METHOD'] = 'elecsys'
```

```
..
```

In questo caso tutte le osservazioni del dataset vengono associate al metodo **Elecsys (file specific)**, consentendo di tracciare in modo esplicito la tecnologia utilizzata per ottenere le misure biologiche. Tale informazione risulta particolarmente importante quando si integrano dati provenienti da dataset diversi o da metodologie analitiche differenti.

- Successivamente viene eseguito il filtraggio delle variabili:

```
filtered_df = dataCleaner.filter_variables(
```

```

datefix_df,

list(zip_files.keys())[0],

new_var=['VISIT_MONTH', 'METHOD'],

prefix='raw'

)

```

La funzione `filter_variables()` recupera il `file_code` associato al dataset tramite i metadati presenti nel Data Lake e filtra il support file mantenendo soltanto le variabili previste per quel dataset.

Oltre alle variabili presenti nel support file, vengono mantenute anche due colonne aggiuntive specificate manualmente e dipendono dalle operazioni fatte prima che hanno generato nuove variabili (`file specific`)

- Infine viene costruito un nuovo DataFrame contenente esclusivamente le colonne selezionate:

```
df_new = df[columns_to_keep]
```

Il risultato è un dataset armonizzato, coerente con i metadati definiti nel support file e arricchito dalle informazioni temporali e metodologiche necessarie per le analisi successive.

In sintesi:

Il codice associa a ogni osservazione il metodo di misura (`file specific`) tramite la variabile `METHOD` e successivamente filtra il DataFrame mantenendo soltanto le variabili previste nel support file, oltre alle colonne aggiuntive `VISIT_MONTH` e `METHOD`, ottenendo un dataset armonizzato e pronto per le successive fasi di elaborazione.

- ☐ **Armonizzare la nomenclatura delle variabili del dataset**, rinominando le colonne secondo gli standard definiti nel support file aggiornato e sincronizzando contemporaneamente i metadati.

- Viene specificato il percorso del nuovo support file:

```
new_support_file_path = 'ADNI_variables_cleaned1'
```

Questo file contiene la versione aggiornata dei metadati, incluse le associazioni tra i nomi originali delle variabili (`orig_variable_code`) e i nomi standardizzati (`variable_code`) da utilizzare nella pipeline di armonizzazione.

- Successivamente viene eseguita la funzione:

```
renamed_df, new_support_file = dataCleaner.new_variable_names(
```

```

new_support_file_path + '.xlsx',
filtered_df,
file_code
)

```

- **new_variable_names(#generica)** → La funzione carica il support file e seleziona le righe corrispondenti al file_code del dataset corrente. In seguito verifica che tutte le colonne presenti nel DataFrame siano documentate nel support file; eventuali variabili non ancora registrate vengono aggiunte automaticamente ai metadati.
- Una volta effettuato questo controllo, viene costruita una corrispondenza tra:

```
orig_variable_code → variable_code
```

che viene utilizzata per rinominare le colonne del DataFrame.

Ad esempio:

```

PTGENDER → GENDER
PTEDUCAT → EDUCATION
PTMARRY → MARRY

```

- La rinomina viene applicata tramite:

```

df.rename(columns=rename_dict)
..

```

ottenendo un dataset con nomi di variabili uniformi e coerenti con quelli utilizzati nell'intera pipeline di elaborazione.

- La funzione restituisce due oggetti:

```

df.rename(columns=rename_dict)
..

```

- contenente il DataFrame con le colonne rinominate, e

```

new_support_file
..

```

contenente il support file eventualmente aggiornato con le nuove variabili individuate durante il processo.

Questa fase rappresenta uno dei passaggi fondamentali dell'armonizzazione dei dati,

poiché permette di integrare dataset provenienti da fonti diverse utilizzando una nomenclatura comune e standardizzata.

In sintesi:

Il codice utilizza il support file aggiornato per rinominare le colonne del dataset secondo una nomenclatura standard condivisa, aggiornando contemporaneamente i metadati con eventuali variabili mancanti e producendo una versione armonizzata del DataFrame pronta per le successive fasi di elaborazione.

- **#specifica** → Fase avanzata di **feature engineering sui biomarcatori** e successivamente costruisce il **profilo biologico ATN**, ampiamente utilizzato nella ricerca sulla malattia di Alzheimer per classificare i soggetti in base alla presenza di deposizione amiloide (A), patologia tau (T) e neurodegenerazione (N).

- Per prima cosa viene applicata la funzione:

```
AbT_df, ratios_var = dataCleaner.get_abeta_tau_ratios(renamed_df)
```

- **get_abeta_tau_ratios** (#specifica) → Questa funzione genera nuove variabili derivate a partire dai biomarcatori presenti nel dataset. In particolare, calcola alcuni rapporti ritenuti particolarmente informativi dal punto di vista clinico e biologico, come:

```
AB4240_CSF = AB42_CSF / AB40_CSF
```

```
TTAU_AB42_CSF = TTAU_CSF / AB42_CSF
```

```
PT181_AB42_CSF = PT181_CSF / AB42_CSF
```

```
PT217_AB42_PL = PT217_PL / AB42_CSF
```

- Tali rapporti costituiscono biomarcatori compositi che spesso mostrano una maggiore capacità discriminante rispetto alle singole misure assolute. Le nuove colonne vengono aggiunte al DataFrame e i loro nomi vengono salvati nella lista:

```
ratios_var
```

- Successivamente il DataFrame arricchito viene utilizzato per calcolare il profilo ATN:

```
final_df, ATN_var = dataCleaner.get_ATN_profile(  
    AbT_df,  
    "cutoffs.json"  
)
```

- La funzione `get_ATN_profile(#specifico)` utilizza il file:

```
cutoffs.json
```

contenente i valori soglia (cutoff) associati ai diversi biomarcatori e ai relativi metodi di misura.

- Per ogni soggetto, i valori dei biomarcatori vengono confrontati con i cutoff appropriati al fine di determinare lo stato:

```
A+ oppure A-
```

```
T+ oppure T-
```

```
N+ oppure N-
```

Ad esempio:

```
AB42 < cutoff → A+
```

```
PTAU > cutoff → T+
```

```
TAU > cutoff → N+
```

- La funzione integra le informazioni disponibili e genera nuove variabili che descrivono il profilo biologico del soggetto, come ad esempio:

```
Apositive
```

```
Tpositive
```

```
Npositive
```

```
ATN_PROFILE
```

oppure altre varianti analoghe previste dalla pipeline.

- Il risultato finale viene salvato nel DataFrame:

```
final_df
```

che contiene sia i biomarcatori originali, sia i rapporti derivati, sia le nuove classificazioni ATN.

- La lista:

```
ATN_var
```

contiene invece i nomi delle variabili create durante la costruzione del profilo ATN.

Questa fase rappresenta uno dei passaggi più importanti dell'intero workflow, poiché trasforma misure biologiche grezze in indicatori clinicamente interpretabili e direttamente

utilizzabili nelle successive analisi statistiche e nei modelli predittivi.

In sintesi:

Il codice genera biomarcatori derivati calcolando rapporti tra le principali misure di amiloide e tau e successivamente utilizza specifici cutoff diagnostici per costruire il profilo biologico ATN di ciascun soggetto, producendo nuove variabili che descrivono lo stato di amiloide (A), tau (T) e neurodegenerazione (N).

❑ **#specifo** → Il blocco di codice si occupa di preparare un DataFrame (datefix_df) prima di passarlo a una fase di filtraggio delle variabili.

- La prima parte è un controllo condizionale basato sul valore della variabile file_code:

```
if file_code == 'UCSFFSL':
```

- Se questo valore è uguale alla stringa 'UCSFFSL', il codice imposta la colonna FSVERSION del DataFrame al valore '4.4':

```
datefix_df['FSVERSION'] = '4.4'
```

- che presumibilmente indica la versione del software FreeSurfer usata per generare i dati (FreeSurfer è un software di analisi di risonanze magnetiche cerebrali, e 'UCSF' suggerisce che questi dati provengano da un centro specifico, l'Università della California San Francisco). Subito dopo, la colonna FLDSTRENG — che probabilmente rappresenta l'intensità del campo magnetico dello scanner MRI (es. "1.5" o "3" Tesla) — viene convertita in stringa e le viene concatenata la lettera 'T':

```
datefix_df['FLDSTRENG'] = datefix_df['FLDSTRENG'].astype(str) + 'T'
```

in modo da ottenere valori come "1.5T" o "3T".

- Se invece file_code non corrisponde a 'UCSFFSL', il codice esegue il ramo else:

```
else:
```

```
    datefix_df['FSVERSION'] = '5.1'
```

che si limita ad assegnare alla colonna FSVERSION il valore '5.1', senza toccare FLDSTRENG. In sostanza, questa logica serve a distinguere due fonti o formati di dati diversi, assegnando la versione corretta del software di elaborazione e, nel primo caso, formattando correttamente l'unità di misura del campo magnetico.

- Dopo questa fase di "correzione" dei dati, il codice richiama un metodo chiamato filter_variables, appartenente a un oggetto dataCleaner (presumibilmente una classe personalizzata per la pulizia dei dati):

```
processed_df = dataCleaner.filter_variables(datefix_df, list(zip_files.keys())[0],  
new_var=['VISIT_MONTH', 'FSVERSION'], prefix='raw')
```

Questo metodo riceve quattro argomenti:

1. `datefix_df` — il DataFrame appena modificato, su cui applicare il filtraggio;
 2. `list(zip_files.keys())[0]` — il primo elemento (chiave) di un dizionario chiamato `zip_files`, convertito prima in lista; questo probabilmente rappresenta un identificativo o il nome del file/dataset di origine, usato dal metodo per sapere quali variabili tenere;
 3. `new_var=['VISIT_MONTH', 'FSVERSION']` — una lista di nomi di colonne che si vogliono aggiungere o considerare come "nuove variabili" rispetto a un set standard, includendo il mese della visita e la versione di FreeSurfer appena impostata;
 4. `prefix='raw'` — un prefisso, probabilmente usato dal metodo per identificare o rinominare un gruppo di colonne originali (grezze, "raw") rispetto ad altre già elaborate.
- Il risultato dell'operazione viene salvato in `processed_df`, che presumibilmente conterrà solo le colonne rilevanti/filtrate, incluse quelle appena create o modificate (FSVERSION, FLDSTRENG se applicabile, VISIT_MONTH).

In sintesi:

Questo frammento fa parte di una pipeline di pulizia dati per un dataset di neuroimaging (verosimilmente relativo a risonanze magnetiche processate con FreeSurfer), e serve a normalizzare le informazioni sulla versione del software e sull'intensità del campo magnetico prima di procedere a un filtraggio mirato delle colonne utili per le analisi successive.

- `#specifico` → Questo blocco prosegue la pipeline di pulizia dati partendo dal `processed_df` ottenuto in precedenza, occupandosi ora di rinominare le variabili e applicare un filtro di qualità
- Per prima cosa viene definito il percorso di un file di supporto che contiene la mappatura per la rinomina delle variabili:

```
new_support_file_path = 'ADNI_variables_cleaned1'
```

Questo nome fa pensare a un dataset ADNI (Alzheimer's Disease Neuroimaging Initiative), un noto studio di neuroimaging longitudinale, e il file `.xlsx` associato probabilmente contiene una tabella di corrispondenza tra i nomi "grezzi" delle variabili e i nomi standardizzati da usare nel dataset finale.

- Subito dopo viene chiamato il metodo `new_variable_names` dell'oggetto `dataCleaner`:

```
renamed_df, new_support_file =  
dataCleaner.new_variable_names(new_support_file_path+'.xlsx', processed_df,  
file_code)
```

Questa funzione riceve il percorso del file Excel di supporto (con estensione aggiunta

concatenando '.xlsx'), il DataFrame `processed_df` da rinominare, e il `file_code` che identifica la fonte/formato dei dati (come visto nel blocco precedente, ad esempio 'UCSFFSL'). Il metodo restituisce due oggetti: `renamed_df`, ovvero il DataFrame con le colonne rinominate secondo lo standard, e `new_support_file`, che probabilmente è una versione aggiornata o filtrata del file di supporto stesso (forse per tenere traccia di quali variabili sono state effettivamente mappate).

- A questo punto viene applicato un filtro relativo ai parametri di controllo qualità (Quality Check) per le immagini segmentate:

```
final_df = dataCleaner.volume_quality_filter(renamed_df)
```

Il metodo `volume_quality_filter` prende in input il DataFrame appena rinominato e restituisce `final_df`, presumibilmente escludendo i record (righe) che non superano determinati criteri di qualità volumetrica delle segmentazioni cerebrali. Nel codice è presente anche un commento dell'autore che segnala un punto da verificare: `##` verificare nomi variabili QC standard, se differiscono capire se metterlo post rename — cioè bisogna controllare se i nomi delle variabili di QC (Quality Check) usati dalla funzione corrispondono a quelli standard, e se necessario capire se questo filtro andrebbe applicato dopo la rinomina anziché prima.

- Nella parte finale, il codice rimuove le colonne relative al controllo qualità, dato che non sono più necessarie una volta completato il filtraggio. Prima viene creata una lista delle colonne che contengono la sottostringa 'QC' nel nome:

```
qc_col = [x for x in final_df if 'QC' in x]
```

Questa è una list comprehension che itera sui nomi delle colonne di `final_df` (iterare su un DataFrame di pandas restituisce di default i nomi delle colonne) e seleziona solo quelle il cui nome contiene la stringa 'QC'.

- Queste colonne vengono poi eliminate dal DataFrame finale:

```
final_df = final_df.drop(columns=qc_col)
```

E, in modo coerente, vengono rimosse anche le righe corrispondenti dal file di supporto `new_support_file`, così da mantenere allineate le due strutture dati:

```
new_support_file = new_support_file[~((new_support_file['file_code'] == file_code) & (new_support_file['variable_code'].isin(qc_col)))]
```

Questa riga usa un filtro booleano negato (`~`): seleziona tutte le righe di `new_support_file` che **non** soddisfano contemporaneamente le due condizioni: avere `file_code` uguale al `file_code` corrente, e avere `variable_code` presente nella lista `qc_col`. In pratica, elimina dal file di supporto le voci relative alle variabili di QC appena scartate, ma solo per la fonte dati corrente (senza toccare eventuali righe relative ad altri `file_code`).

- Infine, viene stampato un messaggio di controllo che mostra il numero di righe prima e dopo il filtraggio, insieme alla lista delle colonne QC rimosse:

```
print('before', len(renamed_df), '\nafter', len(final_df), '\n', qc_col)
```

In sintesi:

Questo frammento completa la pipeline standardizzando i nomi delle variabili secondo una mappatura esterna, applica un filtro di qualità sui volumi segmentati, e infine ripulisce sia il dataset che il file di supporto dalle colonne di controllo qualità ormai superflue, stampando a video un riepilogo dell'effetto del filtraggio.

☐ #specifico →

☐ #specifico → Questo blocco elabora un altro DataFrame di partenza, datefix_df, occupandosi principalmente di dati demografici/anagrafici dei partecipanti (età, genere, stato civile, istruzione, etnia, razza, diagnosi) e di una successiva pulizia dei campi relativi a intensità di campo magnetico e versione FreeSurfer.

- Per prima cosa viene creata una copia profonda del DataFrame di origine, così da non modificare accidentalmente datefix_df:

```
processed_df = datefix_df.copy(deep=True)
```

- Subito dopo, la colonna AGE viene rinominata in AGE_bl (dove "bl" sta presumibilmente per *baseline*, cioè l'età registrata alla visita iniziale/basale):

```
processed_df.rename(columns={'AGE': 'AGE_bl'}, inplace=True)
```

- Viene poi creata una nuova colonna AGE, calcolata riga per riga tramite una funzione lambda applicata con apply:

```
processed_df['AGE'] = processed_df.apply(lambda row:  
dataCleaner.add_calculated_age(exam_date=row['EXAMDATE'], age_bl=row['AGE_bl'],  
bl_date=row['EXAMDATE_bl']), axis=1)
```

Il metodo add_calculated_age riceve la data dell'esame corrente (EXAMDATE), l'età basale (AGE_bl) e la data dell'esame basale (EXAMDATE_bl), e presumibilmente calcola l'età effettiva del partecipante al momento della visita corrente, sommando all'età basale la differenza temporale tra le due date. In questo modo si ottiene un'età aggiornata per ogni singola visita, anziché mantenere solo quella iniziale.

- Le righe successive applicano una serie di trasformazioni categoriche a diverse variabili demografiche, ciascuna tramite un metodo dedicato di dataCleaner:

```
processed_df = dataCleaner.binarization_gender(processed_df, col_name='PTGENDER')  
processed_df = dataCleaner.categorize_marry(processed_df, col_name='PTMARRY')  
processed_df = dataCleaner.categorize_education(processed_df, col_name='PTEDUCAT')  
processed_df = dataCleaner.categorize_ethnicity(processed_df, col_name='PTETHCAT')  
processed_df = dataCleaner.categorize_race(processed_df, col_name='PTRACCAT')
```

```
processed_df = dataCleaner.categorize_diagnosis(processed_df, col_name='DX')
```

Funzioni nello specifico: `binarization_gender` trasforma la colonna del genere (PTGENDER) in una forma binaria (ad esempio 0/1); `categorize_marry` categorizza lo stato civile (PTMARRY); `categorize_education` raggruppa il livello di istruzione (PTEDUCAT) in categorie; `categorize_ethnicity` e `categorize_race` fanno lo stesso per etnia (PTETHCAT) e razza (PTRACCAT); infine `categorize_diagnosis` categorizza la diagnosi clinica (DX), verosimilmente distinguendo tra condizioni come cognitivamente normale, decadimento cognitivo lieve e demenza/Alzheimer, coerentemente con un dataset ADNI.

- Successivamente viene richiamato di nuovo il metodo di filtraggio delle variabili già visto in precedenza:

```
processed_df = dataCleaner.filter_variables(processed_df, list(zip_files.keys())[0], new_var=['AGE_bl', 'VISIT_MONTH'], prefix='raw')
```

In questo caso le "nuove variabili" da considerare sono AGE_bl (l'età basale, creata rinominando la colonna originale) e VISIT_MONTH. È presente anche un commento dell'autore che riflette un dubbio aperto: `#in this case i might like to delate AGE_bl since it is now substituted by AGE` — cioè si sta considerando se rimuovere AGE_bl, dato che ormai è stata sostituita a livello informativo dalla nuova colonna AGE calcolata per ogni visita.

- Infine, le ultime due righe puliscono il formato dei campi FLDSTRENG e FSVERSION, presumibilmente per uniformarli con l'altro ramo della pipeline visto in precedenza (quello con `file_code == 'UCSFFSL'`):

```
processed_df['FLDSTRENG'] = processed_df['FLDSTRENG'].str.extract('([0-9.]+)', expand=False)+'T'
```

Qui viene usato `str.extract` con un'espressione regolare `('([0-9.]+)')` per estrarre solo la parte numerica (cifre e punto decimale) dal valore testuale di FLDSTRENG, scartando eventuali altri caratteri; il parametro `expand=False` fa sì che il risultato sia una Serie (e non un DataFrame). Al valore numerico estratto viene poi concatenata la lettera 'T', ottenendo di nuovo un formato tipo "3T" o "1.5T".

- In modo analogo, anche FSVERSION viene ripulita estraendo solo la parte numerica tramite la stessa espressione regolare, senza però aggiungere alcun suffisso:

```
processed_df['FSVERSION'] = processed_df['FSVERSION'].str.extract('([0-9.]+)', expand=False)
```

In sintesi:

Questo frammento arricchisce il dataset con informazioni anagrafiche più accurate (in particolare un'età calcolata dinamicamente per ogni visita), standardizza numerose variabili demografiche in categorie coerenti, e infine normalizza il formato dei campi tecnici FLDSTRENG e FSVERSION estraendone solo la componente numerica rilevante.

- #specifico → Applica una serie di trasformazioni categoriche a variabili demografiche e cliniche, ma partendo direttamente da `datefix_df` e con nomi di colonna diversi rispetto al blocco precedente — probabilmente perché qui si sta lavorando su una fonte dati differente (le colonne con prefisso `PHC_` suggeriscono un formato tipo "Phenotype Harmonization Consortium", spesso usato in dataset armonizzati come quelli di ADNI o studi collegati).

- Il primo passaggio binarizza la colonna relativa al sesso del partecipante:

```
processed_df = dataCleaner.binarization_gender(datefix_df, col_name='PHC_Sex')
```

Qui il metodo `binarization_gender` riceve direttamente `datefix_df` come input (e non una sua copia esplicita, a differenza del blocco precedente dove si usava `.copy(deep=True)`), applicando la trasformazione sulla colonna `PHC_Sex` e restituendo il risultato in `processed_df`.

- Successivamente viene categorizzato il livello di istruzione:

```
processed_df = dataCleaner.binarization_gender(datefix_df, col_name='PHC_Sex')
```

che agisce sulla colonna `PHC_Education`, raggruppando i valori in categorie (analogamente a quanto visto in precedenza per `PTEDUCAT`).

- Poi viene categorizzata l'etnia:

```
processed_df = dataCleaner.categorize_ethnicity(processed_df, col_name='PHC_Ethnicity')
```

applicata qui alla colonna `PHC_Ethnicity`.

- La riga successiva introduce una trasformazione diversa dalle precedenti, ovvero la creazione di variabili dummy (variabili binarie indicatrici) a partire da una classificazione clinica:

```
processed_df, new_var = dataCleaner.convert_to_dummies_ATNC_profile(processed_df, col_name='AT_class')
```

Il metodo `convert_to_dummies_ATNC_profile` prende la colonna `AT_class` — che presumibilmente rappresenta un profilo biologico secondo la classificazione ATN (Amyloid-Tau-Neurodegeneration), un framework diagnostico usato negli studi sull'Alzheimer per classificare i pazienti in base alla presenza di biomarcatori di amiloide (A), tau (T) e neurodegenerazione (N) — e la converte in più colonne binarie "dummy", una per ciascuna categoria/profilo possibile (ad esempio `A+T+N+`, `A+T-N-`, ecc.). A differenza dei metodi precedenti, questa funzione restituisce due valori: il DataFrame aggiornato `processed_df` con le nuove colonne dummy aggiunte, e `new_var`, che è probabilmente la lista dei nomi di queste nuove colonne appena create, utile poi per tenerne traccia (ad esempio in una successiva chiamata a `filter_variables`, come visto nei blocchi precedenti).

- Infine, viene categorizzata la diagnosi clinica:

```
processed_df = dataCleaner.categorize_diagnosis(processed_df,
col_name='PHC_Diagnosis')
```

applicata alla colonna PHC_Diagnosis, analogamente a quanto fatto in precedenza sulla colonna DX in un altro ramo della pipeline.

In sintesi →

Questo frammento di codice replica concettualmente le trasformazioni demografiche e cliniche già viste (sesso, istruzione, etnia, diagnosi), ma adattandole ai nomi di colonna di una fonte dati diversa (con prefisso PHC_), e aggiunge in più la creazione di variabili dummy a partire da una classificazione biologica ATN, un'informazione tipicamente rilevante per la ricerca su Alzheimer e biomarcatori.

- ❑ **#specifico** → Questo breve blocco di codice interviene per correggere un problema segnalato nel commento iniziale:

```
# non funziona la funzione perche black e Native Hawaiian or PI sono invertite
```

Il commento chiarisce che una funzione di categorizzazione della razza (presumibilmente `categorize_race`, vista in un blocco precedente, o una sua variante) non funziona correttamente per questa fonte dati, perché le categorie "Black" e "Native Hawaiian or Pacific Islander" risultano invertite tra loro nella codifica originale. Per questo motivo, invece di richiamare la funzione automatica, viene applicata una correzione manuale tramite una mappatura esplicita dei codici.

- Viene quindi definito un dizionario di mappatura:

```
mapping = {1: 1, 1.0: 1, 2: 2, 2.0: 2, 4: 3, 4.0: 3, 3: 4, 3.0: 4, 5: 5, 5.0: 5}
```

Questo dizionario associa ad ogni valore originale della colonna PHC_Race (sia come intero che come float, dato che nei dati potrebbero comparire in entrambe le forme, ad esempio 2 oppure 2.0) un nuovo valore corretto. Osservando i numeri, si nota che le coppie 1→1, 2→2 e 5→5 restano invariate, mentre i valori 4 e 3 vengono scambiati tra loro (4→3 e 3→4): questo è esattamente lo scambio necessario per correggere l'inversione tra le categorie "Black" e "Native Hawaiian or Pacific Islander" menzionata nel commento, presumendo che questi due codici corrispondano proprio a quelle categorie nella codifica originale.

- Infine, questa mappatura viene applicata alla colonna PHC_Race del DataFrame tramite il metodo `.map()`:

```
processed_df['PHC_Race'] = processed_df['PHC_Race'].map(mapping)
```

Il metodo `.map()` sostituisce ogni valore della colonna con il corrispondente valore definito nel dizionario `mapping`, aggiornando così PHC_Race con i codici corretti (eventuali valori non presenti come chiave nel dizionario verrebbero invece trasformati in NaN).

In sintesi:

Questo frammento rappresenta una correzione "manuale" e mirata, resa necessaria da un bug nella funzione di categorizzazione automatica: invece di affidarsi a `categorize_race`, si ridefinisce esplicitamente la corrispondenza tra i vecchi codici numerici e quelli corretti, risolvendo così l'inversione tra le categorie "Black" e "Native Hawaiian or Pacific Islander".

- ❑ `#specifico` → Questo blocco si occupa della colonna relativa al genere, ma questa volta con un nome di colonna diverso rispetto ai blocchi precedenti: `GENDER` invece di `PTGENDER` o `PHC_Sex`, segno che si sta lavorando su un'ulteriore fonte dati con una propria nomenclatura.

- Per prima cosa, il nome della colonna viene salvato in una variabile:

```
col_name = 'GENDER'
```

questo probabilmente per rendere il codice più leggibile e riutilizzabile nelle righe successive, evitando di ripetere la stringa `'GENDER'` più volte.

- Successivamente, come chiarito dal commento `#` mi assicuro che tutta la colonna sia di interi, viene forzata la conversione dell'intera colonna al tipo intero a 64 bit:

```
processed_df[col_name] = processed_df[col_name].astype('int64')
```

Questo passaggio serve a garantire uniformità nei dati: se la colonna contenesse valori misti (ad esempio numeri interi salvati come float, tipo 1.0 invece di 1, oppure stringhe numeriche), la conversione esplicita a `int64` elimina queste ambiguità e assicura che tutti i valori siano rappresentati in modo consistente come interi.

- Infine, è presente un commento che descrive l'intenzione originaria ma chiarisce che l'azione non è più necessaria:

```
# mapp gender in to classes: 0 = 'female' → 0, 1 = 'male' → 1 → non  
necessario perchè già nel formato corretto
```

Questo commento spiega che l'idea iniziale era di mappare i valori del genere in classi standardizzate (0 per "female", 1 per "male"), analogamente a quanto fatto con `binarization_gender` nei blocchi precedenti per altre fonti dati. Tuttavia, in questo caso specifico, la colonna `GENDER` risulta già codificata correttamente in questo formato binario (0/1), quindi non è necessaria alcuna trasformazione aggiuntiva oltre alla conversione del tipo di dato — da qui la conclusione "non necessario perché già nel formato corretto".

In sintesi:

Questo frammento è essenzialmente un controllo di coerenza sul tipo di dato della colonna `GENDER`, che viene forzata a intero per sicurezza, mentre la codifica dei valori (0 = female, 1 = male) viene lasciata

invariata poiché già conforme allo standard desiderato, senza bisogno di applicare la funzione di binarizzazione usata altrove nella pipeline.

- ❑ `#specifico` → Questo blocco si occupa della categorizzazione dello stato civile per un'altra fonte dati, identificata questa volta dalla colonna MARISTAT (diversa da PTMARRY vista in un blocco precedente). Il commento iniziale ne indica lo scopo:

```
# CATEGORIZZAZIONE Status Maritale
```

- Il nome della colonna viene salvato in una variabile, come già visto per GENDER:

```
col_name = 'MARISTAT'
```

- La riga successiva è accompagnata dal commento `# mi assicuro che tutta la colonna sia di interi`, ma in realtà non esegue alcuna conversione di tipo:

```
processed_df[col_name] = processed_df[col_name]
```

Questa istruzione riassegna semplicemente la colonna a se stessa, senza alcun effetto pratico: sembra un residuo di codice, probabilmente una riga copiata dal blocco precedente (quello su GENDER) e non aggiornata correttamente — lì infatti c'era `.astype('int64')`, che qui risulta mancante nonostante il commento lo lasci intendere. Di fatto, quindi, questa riga è superflua e non produce alcuna conversione del tipo di dato.

- Successivamente viene definita la mappatura per ricodificare i valori originali dello stato civile in nuove categorie, come descritto nel commento dettagliato:

```
# Map marital status to classes: 1 = 'married' → 1, 6 = 'Living as married' → 1, 3 = 'divorced' → 2, 4 = 'separated' → 2, 2 = 'widowed' → 3, 5 = 'never married' → 0, 7 = 'Other' → nan, 9 = 'Unknown' → nan
```

Il commento spiega la logica di raggruppamento: i codici originali 1 ("married") e 6 ("Living as married") vengono accorpati nella nuova classe 1; i codici 3 ("divorced") e 4 ("separated") vengono accorpati nella classe 2; il codice 2 ("widowed") diventa la classe 3; il codice 5 ("never married") diventa la classe 0; infine i codici 7 ("Other") e 9 ("Unknown") non vengono mappati esplicitamente e diventeranno quindi NaN.

- Questa logica viene tradotta nel dizionario mapping:

```
mapping = {1: 1, 6: 1, 3: 2, 4: 2, 2: 3, 5: 0, 1.0: 1, 6.0: 1, 3.0: 2, 4.0: 2, 2.0: 3, 5.0: 0}
```

Si nota che ogni codice è presente sia in forma intera (es. 1: 1) sia in forma float (es. 1.0: 1), per gestire entrambi i possibili tipi di dato presenti nella colonna originale — coerentemente con quanto fatto nel blocco precedente sulla correzione di PHC_Race. Da notare inoltre che i codici 7 e 9 non compaiono affatto nel dizionario: questo significa che, tramite `.map()`, verranno automaticamente convertiti in NaN, in linea con quanto specificato nel commento ("Other" e "Unknown" devono diventare valori mancanti).

- Infine, la mappatura viene applicata alla colonna:

```
processed_df[col_name] = processed_df[col_name].map(mapping)
```

sostituendo ogni valore originale di MARISTAT con la nuova classe corrispondente definita nel dizionario.

In sintesi:

Questo frammento ricodifica manualmente lo stato civile in quattro categorie semplificate (0 = mai sposato, 1 = sposato/convivente come sposato, 2 = divorziato/separato, 3 = vedovo), trasformando in valori mancanti le categorie residuali "Other" e "Unknown"; contiene però una riga di codice apparentemente inutile (la riassegnazione della colonna a se stessa) che non esegue la conversione di tipo suggerita dal commento sovrastante.

- ❑ **#specifico** → Questo blocco si occupa della categorizzazione dell'etnia, per un'ulteriore fonte dati, questa volta identificata dalla colonna HISPANIC.
- Il nome della colonna viene salvato in una variabile, secondo lo schema già visto negli altri blocchi:

```
col_name = 'HISPANIC'
```

- Come indicato dal commento # mi assicuro che tutta la colonna sia di interi, la colonna viene qui effettivamente convertita al tipo intero a 64 bit (a differenza del blocco precedente su MARISTAT, dove questa conversione era solo annunciata ma non eseguita):

```
processed_df[col_name] = processed_df[col_name].astype('int64')
```

- Successivamente viene applicata una ricodifica dei valori, descritta nel commento:

```
# mapp Ethnicity in to classes: 0 = 'not hisp/latino' → 1, 1 = 'hisp/latino' → 0
```

Il commento chiarisce che si sta effettuando un'inversione della codifica originale: il valore 0 (che nella fonte dati originale significa "not hisp/latino") viene trasformato nel nuovo valore 1, mentre il valore 1 (che significa "hisp/latino") viene trasformato nel nuovo valore 0. Si tratta quindi di un capovolgimento della codifica binaria, probabilmente per allinearla a uno standard usato nel resto della pipeline (dove magari 1 indica l'appartenenza a un gruppo etnico specifico, coerentemente con le altre funzioni categorize_ethnicity viste nei blocchi precedenti).

- Questa trasformazione viene implementata tramite una funzione lambda passata a .map():

```
processed_df[col_name] = processed_df[col_name].map(lambda x: 1 if x in [0, 0.0] else (0 if x in [1, 1.0] else np.nan))
```


La funzione lambda esamina ogni valore x della colonna: se x è 0 oppure 0.0 (gestendo quindi sia la forma intera che quella float, come già visto in altri blocchi), restituisce 1; se invece x è 1 oppure 1.0, restituisce 0; per qualsiasi altro valore (ad esempio codici sconosciuti o già mancanti), restituisce np.nan, marcandolo esplicitamente come dato mancante.

In sintesi:

Questo frammento converte la colonna HISPANIC in formato intero e poi ne inverte la codifica binaria originale (0/1) in una nuova codifica (1/0), gestendo al contempo eventuali valori anomali o non previsti impostandoli a NaN tramite np.nan (che presuppone l'importazione della libreria NumPy nel resto dello script).

❑ **#specifico** → Questo blocco si occupa della categorizzazione della razza per un'altra fonte dati, identificata dalla colonna RACE.

- Il nome della colonna viene salvato in una variabile, secondo lo schema ormai ricorrente:

```
col_name = 'RACE'
```

- Come indicato dal commento # mi assicuro che tutta la colonna sia di interi, la colonna viene convertita al tipo intero a 64 bit:

```
processed_df[col_name] = processed_df[col_name].astype('int64')
```

- Successivamente viene definita la logica di ricodifica, spiegata nel commento:

```
# Map race to classes: 1 = 'White' → 5, 2 = 'Black' → 4, 5 = 'Asian' → 2, 3 = 'Am Indian/Alaskan' → 1, 4 = 'Hawaiian/Other PI' → 3)
```

Il commento chiarisce come ogni categoria originale venga riassegnata a un nuovo codice numerico: 1 ("White") diventa 5; 2 ("Black") diventa 4; 5 ("Asian") diventa 2; 3 ("Am Indian/Alaskan") diventa 1; 4 ("Hawaiian/Other PI") diventa 3. In pratica si tratta di una permutazione completa dei codici originali, probabilmente per allinearli allo stesso standard di codifica usato altrove nella pipeline (ad esempio lo stesso schema numerico visto nel blocco relativo a PHC_Race, dove i codici 1, 2, 3, 4, 5 corrispondevano a categorie specifiche in un ordine standardizzato).

- Questa logica viene tradotta nel dizionario mapping:

```
mapping = {1: 5, 2: 4, 3: 1, 4: 3, 5: 2, 1.0: 5, 2.0: 4, 3.0: 1, 4.0: 3, 5.0: 2}
```

Anche qui, come nei blocchi precedenti, ogni codice è presente sia in forma intera sia in forma float (ad esempio sia 1: 5 che 1.0: 5), per gestire entrambi i possibili tipi di dato presenti nella colonna originale.

- Infine, la mappatura viene applicata alla colonna tramite .map():

```
processed_df[col_name] = processed_df[col_name].map(mapping)
```

sostituendo ogni valore originale di RACE con il nuovo codice corrispondente definito nel dizionario.

In sintesi:

Questo frammento converte la colonna RACE in formato intero e ne riordina completamente la codifica numerica secondo una nuova corrispondenza definita manualmente, verosimilmente per armonizzare questa fonte dati con lo schema di classificazione della razza già utilizzato altrove nella pipeline (coerentemente con la correzione vista in precedenza per PHC_Race).

- **#specifico** → Questo blocco si occupa della categorizzazione di una variabile legata allo studio DIAN (Dominantly Inherited Alzheimer Network, uno studio che coinvolge famiglie portatrici di mutazioni genetiche dominanti associate all'Alzheimer ad esordio precoce).

- Il nome della colonna viene salvato in una variabile:

```
col_name = 'DIAN_GROUP'
```

- Come indicato dal commento # mi assicuro che tutta la colonna sia di interi, la colonna viene convertita al tipo intero a 64 bit:

```
processed_df[col_name] = processed_df[col_name].astype('int64')
```

- Successivamente viene descritta, tramite una serie di commenti, la logica di ricodifica dei gruppi:

```
# Map race to classes:

#0 (ADNI, no DIAN) → nan

# 1 (no mutation present) → 0

# 2 (mutation present but cognitive normal) → 1

# 3 (mutation present andcol_name = 2
```

Da notare che l'intestazione del commento (# Map race to classes:) è probabilmente un residuo copiato dal blocco precedente relativo a RACE e non è stata aggiornata: qui infatti non si sta mappando la razza, ma i gruppi di appartenenza rispetto allo studio DIAN. Il commento spiega che il valore 0 (che identifica i soggetti provenienti da ADNI, quindi non appartenenti allo studio DIAN) dovrebbe diventare NaN; il valore 1 ("no mutation present", cioè nessuna mutazione genetica presente) dovrebbe diventare 0; il valore 2 ("mutation present but cognitive normal", mutazione presente ma con stato cognitivo normale) dovrebbe diventare 1; e il valore 3 dovrebbe diventare 2 — anche se qui il commento risulta troncato e mal formattato (# 3 (mutation present andcol_name = 2), probabilmente per un errore di battitura in cui la spiegazione della categoria 3

(verosimilmente "mutation present and cognitively impaired" o simile) si è accidentalmente fusa con la riga di codice successiva (col_name = 2), rendendo il commento incompleto e poco chiaro nella sua parte finale.

- Infine, la mappatura viene applicata alla colonna:

```
processed_df[col_name] = processed_df[col_name].map(mapping)
```

sostituendo i valori originali di DIAN_GROUP con i nuovi codici corrispondenti.

In sintesi:

Questo frammento ricodifica la variabile di gruppo DIAN in una nuova scala a tre livelli (0 = nessuna mutazione, 1 = mutazione presente ma cognitivamente normale, 2 = mutazione presente con probabile compromissione cognitiva), escludendo implicitamente (tramite valore mancante) i soggetti provenienti da ADNI che non fanno parte dello studio DIAN; il codice presenta però un commento copiato e mal aggiornato dal blocco precedente, oltre a un refuso che rende illeggibile la descrizione dell'ultima categoria.

- **#specifico** → Questo blocco gestisce una situazione comune nei dataset clinici/biologici: la stessa grandezza biologica misurata con **metodi analitici diversi**, che quindi finisce per essere registrata in colonne separate pur rappresentando concettualmente la stessa informazione.
- Viene richiamato il metodo `handel_same_variable_different_methods` dell'oggetto `dataCleaner` (da notare, per inciso, il refuso nel nome del metodo: "handel" invece di "handle"):

```
methods_df = dataCleaner.handel_same_variable_different_methods(  
    df=processed_df,  
    var1=['CSF_ELC_AB42', 'CSF_ELC_PTAU', 'CSF_ELC_TAU', 'CSF_ELC_AB40',  
    'CSF_ELC_AB4240'],  
    var2=['MSP_AB40', 'MSP_AB42'],  
    method1='elecsys', method2='massospectrometry',  
    mapping={'MSP_AB40': 'CSF_ELC_AB40', 'MSP_AB42': 'CSF_ELC_AB42'})
```

Il metodo riceve diversi parametri:

- `df=processed_df` — il DataFrame di partenza su cui operare, quello elaborato nei blocchi precedenti;
- `var1=['CSF_ELC_AB42', 'CSF_ELC_PTAU', 'CSF_ELC_TAU', 'CSF_ELC_AB40', 'CSF_ELC_AB4240']` — un primo gruppo di colonne relative a biomarcatori liquorali (CSF, Cerebrospinal Fluid) misurati con la piattaforma **Elecsys**: si tratta di amiloide beta 42 (AB42), tau fosforilata (PTAU), tau totale (TAU), amiloide beta

40 (AB40) e il rapporto tra le due forme di amiloide (AB42/AB40), presumibilmente il rapporto AB42/AB40);

- `var2=['MSP_AB40', 'MSP_AB42']` — un secondo gruppo di colonne relative agli stessi biomarcatori di amiloide (AB40 e AB42), ma misurati con un metodo diverso, la **spettrometria di massa** (Mass Spectrometry, da cui il prefisso MSP_);
- `method1='elecsys'` e `method2='massospectrometry'` — le etichette testuali che identificano i due metodi analitici corrispondenti rispettivamente a `var1` e `var2`, presumibilmente usate dalla funzione per tracciare la provenienza di ciascun valore (ad esempio in una colonna aggiuntiva che indica quale metodo è stato usato riga per riga);
- `mapping={'MSP_AB40':'CSF_ELC_AB40', 'MSP_AB42':'CSF_ELC_AB42'}` — un dizionario che stabilisce la corrispondenza tra le colonne del secondo metodo e quelle "equivalenti" del primo metodo, indicando cioè che `MSP_AB40` corrisponde concettualmente a `CSF_ELC_AB40` e `MSP_AB42` corrisponde a `CSF_ELC_AB42`.
- Il risultato dell'operazione viene salvato in `methods_df`. Sulla base dei parametri passati, la funzione probabilmente unifica le misurazioni provenienti dai due metodi analitici in un'unica colonna per ciascun biomarcatore condiviso (ad esempio combinando `CSF_ELC_AB40` e `MSP_AB40` in un'unica colonna `AB40`, dando priorità a un metodo sull'altro oppure integrando i valori mancanti dell'uno con quelli disponibili dell'altro), aggiungendo eventualmente anche un'indicazione di quale metodo sia stato effettivamente usato per ciascun valore finale — ma senza vedere l'implementazione della funzione, questo rimane un'ipotesi plausibile basata sui parametri passati.

In sintesi:

Questo frammento affronta il problema dell'armonizzazione di misure biologiche duplicate (in questo caso i livelli di amiloide beta nel liquido cerebrospinale) ottenute con due tecniche di laboratorio differenti — Elecsys e spettrometria di massa — mappando esplicitamente quali colonne del secondo metodo corrispondono a quali colonne del primo, presumibilmente per produrre un dataset finale con valori consolidati e non duplicati.

- ☐ **Analizzare la distribuzione dei principali biomarcatori liquorali (CSF) e dei relativi rapporti derivati**, calcolandone il **1° percentile** e il **99° percentile**. Queste statistiche sono particolarmente utili per identificare valori estremi e definire intervalli di riferimento da utilizzare nelle successive fasi di controllo qualità e normalizzazione dei dati.

- Viene definito l'elenco delle variabili di interesse:

```
var_of_interest = [  
    'AB40-CSF',
```

```
'AB42_CSF',  
'TTAU_CSF',  
'PT181_CSF',  
'AB4240_CSF',  
'TTAU_AB42_CSF',  
'PT181_AB42_CSF'  
]
```

L'elenco include sia biomarcatori misurati direttamente sia biomarcatori derivati ottenuti come rapporti tra le misure di amiloide e tau.

- Successivamente viene creato un nuovo DataFrame contenente solo queste variabili:

```
df_interest = final_df[var_of_interest].copy(deep=True)
```

L'utilizzo di una copia profonda garantisce che eventuali elaborazioni successive non modifichino il dataset originale.

- Viene quindi inizializzata una tabella che conterrà i percentili calcolati:

```
percentile_df = pd.DataFrame(  
    index=df_interest.keys(),  
    columns=['1° percentile', '99° percentile']  
)
```

Le righe rappresentano le variabili da analizzare, mentre le colonne conterranno i valori del primo e del novantanovesimo percentile.

- Successivamente il codice scorre tutte le variabili selezionate:

```
for var in df_interest.keys():
```

- Per ciascuna variabile vengono calcolati:

```
p1 = df_interest[var].quantile(0.01)  
p99 = df_interest[var].quantile(0.99)
```

ovvero:

- il **1° percentile**, che identifica il valore sotto il quale cade circa l'1% delle osservazioni;

- il **99° percentile**, che identifica il valore sopra il quale cade circa l'1% delle osservazioni.

I valori vengono poi arrotondati a quattro cifre decimali e salvati nella tabella riassuntiva:

```
percentile_df.loc[var, '1° percentile'] = round(p1, 4)

percentile_df.loc[var, '99° percentile'] = round(p99, 4)
```

- Infine viene stampato un riepilogo dei risultati:

```
print("1 e 99 percentili variabili CSF → metodo ELECSYS")

print(percentile_df.dropna(how='all'))
```

La funzione `dropna(how='all')` elimina eventuali righe completamente vuote, mostrando solamente le statistiche effettivamente calcolate.

Il risultato è una tabella che riassume l'intervallo osservato per ciascun biomarcatore, escludendo gli outlier più estremi e fornendo una descrizione robusta della distribuzione dei dati ottenuti mediante la piattaforma analitica **Elecsys**.

In sintesi:

Il codice seleziona i principali biomarcatori liquorali e i relativi rapporti derivati, calcola per ciascuno il 1° e il 99° percentile e produce una tabella riassuntiva utile per descrivere la distribuzione dei dati, identificare valori estremi e supportare le successive fasi di controllo qualità e normalizzazione.

- ☐ Semplice analisi descrittiva del dataset finale per valutare la numerosità dei soggetti inclusi e la disponibilità di dati longitudinali.
- Per prima cosa viene calcolato il numero totale di soggetti unici presenti nel dataset utilizzando la variabile identificativa RID:

```
tot_sub = len(final_df['RID'].unique())
```

La funzione `unique()` restituisce tutti gli identificativi distinti dei soggetti presenti nel DataFrame, mentre `len()` ne conta il numero totale.

- Il risultato viene quindi visualizzato:

```
print('totale subject:', tot_sub)
```

ottenendo il numero complessivo di individui disponibili per le analisi.

- Successivamente viene valutato quanti soggetti dispongano di più di una visita:

```
multiple_visits = (
```

```
final_df['RID'].value_counts() > 1  
) .sum()
```

In questo passaggio:

- `value_counts()` conta quante osservazioni sono disponibili per ciascun soggetto;
- il confronto `> 1` identifica i soggetti con almeno due visite;
- `sum()` conta quanti soggetti soddisfano questa condizione.
- Infine viene stampato il risultato:

```
print('subjects with multiple visits: ', multiple_visits)
```

Questo indicatore è particolarmente importante nei dataset longitudinali, poiché quantifica il numero di soggetti per cui è possibile studiare l'evoluzione temporale dei biomarcatori e delle caratteristiche cliniche.

In sintesi:

Il codice calcola il numero totale di soggetti presenti nel dataset e determina quanti di essi dispongano di più di una visita, fornendo una misura della dimensione del campione e della disponibilità di dati longitudinali per le successive analisi.

- ☐ **Fase finale di aggiornamento e salvataggio del support file**, nella quale vengono incorporate le informazioni statistiche delle variabili generate durante tutte le trasformazioni effettuate sul dataset.

- Per prima cosa il support file viene sincronizzato con il DataFrame finale:

```
new_support_file = update_variables_support_file(  
    final_df,  
    new_support_file,  
    file_code  
)
```

- `update_variables_support_file` (#generica) → La funzione confronta le variabili presenti nel dataset finale (`final_df`) con quelle già registrate nel support file e, se necessario, aggiunge le nuove variabili generate durante il preprocessing, come ad esempio:

AB4240_CSF

TTAU_AB42_CSF

PT181_AB42_CSF

Apositive

Tpositive

Npositive

In questo modo il support file rimane coerente con la struttura effettiva del dataset.

- Successivamente viene creato un nuovo oggetto InfoSupportFile:

```
infoSupportFile = InfoSupportFile(  
    new_support_file,  
    final_df,  
    file_name  
)
```

che permette di analizzare ciascuna variabile presente nel DataFrame e aggiornare i relativi metadati.

- Il codice percorre quindi tutte le colonne del dataset finale:

```
for key in final_df.keys():
```

e verifica se la variabile è documentata nel support file per il file_code corrente:

```
if key in new_support_file[  
    new_support_file['file_code'] == file_code  
][ 'variable_code' ].values:
```

- Per ogni variabile valida viene richiamata la funzione:

```
new_support_file = infoSupportFile.get_variable_info(key)
```

- `get_variable_info` (#generica) → questa funzione aggiorna automaticamente nel support file diverse informazioni descrittive e di qualità, tra cui:
 - tipo della variabile (type_variable);
 - classi presenti (classes);
 - intervallo dei valori (range);
 - numero di osservazioni valide (valid_values);
 - numero di valori mancanti (missing_values);
 - popolazioni ADNI in cui la variabile è assente (missing_pop);
 - indicazione di mantenimento o eliminazione (keep / drop).

In questo modo anche le variabili derivate generate durante la pipeline vengono documentate con lo stesso livello di dettaglio delle variabili originali.

- Infine il support file aggiornato viene salvato su disco:

```
save_df(  
    new_support_file,  
    new_support_file_path  
)
```

producendo una versione finale dei metadati completamente sincronizzata con il dataset elaborato.

In sintesi:

Il codice aggiorna il support file con tutte le variabili presenti nel dataset finale, incluse quelle generate durante il preprocessing, ne calcola automaticamente le principali statistiche descrittive e di qualità (tipo, classi, range, valori mancanti e copertura delle popolazioni) e salva infine una versione aggiornata e completa dei metadati del dataset.

- ☐ Raccoglie automaticamente alcune informazioni necessarie per la fase finale di esportazione e versionamento del dataset armonizzato.
- Per prima cosa viene identificato l'insieme delle popolazioni ADNI effettivamente presenti nel dataset:

```
lst_population = infoSupportFile.get_present_populations()
```

- `get_present_populations` (#generico) → La funzione `get_present_populations()` utilizza le informazioni contenute nel support file, in particolare la colonna `missing_pop`, per determinare quali coorti ADNI risultano rappresentate nei dati. Il risultato è una lista contenente le popolazioni effettivamente disponibili, ad esempio:

```
['ADNI1', 'ADNI2', 'ADNI3']
```

Questa informazione verrà successivamente utilizzata per compilare i metadati del file e documentare la copertura del dataset.

- Successivamente viene generato automaticamente il nome della nuova versione del file:

```
new_file_name = dataCleaner.file_versions_name(  
    file_name=file_name,
```

```
suffix='_01'
```

```
)
```

- `file_versions_name` (#generica) → costruisce un nuovo nome a partire dal nome del file originale, aggiungendo un suffisso che identifica la versione corrente del dataset.

Ad esempio:

```
UPENNBIOIMK_ROCHE_ELECSYS.csv
```

può diventare:

```
UPENNBIOIMK_ROCHE_ELECSYS_01.csv
```

oppure:

```
UPENNBIOIMK_ROCHE_ELECSYS_L1_01.csv
```

a seconda delle regole implementate nella funzione.

Questa operazione consente di mantenere una corretta gestione delle versioni dei dataset prodotti dalla pipeline, evitando sovrascritture accidentali e facilitando la tracciabilità delle trasformazioni applicate ai dati.

In sintesi:

Il codice identifica automaticamente le popolazioni ADNI effettivamente rappresentate nel dataset attraverso le informazioni contenute nel support file e genera il nome della nuova versione del file elaborato, preparando il dataset alle successive fasi di salvataggio e pubblicazione.

- ☐ Fase conclusiva della pipeline di elaborazione, durante la quale il dataset armonizzato viene **caricato nel Data Lake** insieme ai metadati necessari per la sua identificazione e tracciabilità.

- L'upload viene eseguito tramite la funzione:

```
result = client.upload_dataframe(  
    df=final_df,  
    object_name=new_file_name,  
    prefix='cleaned/single_file/',
```

```
metadata={
    'population': lst_population,
    'level': 'cleaned_01',
    'file_code': file_code,
    'source': 'ADNI'
}
```

- Il DataFrame da salvare è:

```
df=final_df
```

ovvero il dataset ottenuto al termine di tutte le operazioni di pulizia, armonizzazione, calcolo dei biomarcatori derivati e classificazione ATN.

- Il parametro:

```
object_name=new_file_name
```

specifica il nome assegnato al file all'interno del Data Lake. Tale nome è stato generato automaticamente nella fase precedente per garantire un corretto versionamento del dataset.

- La cartella di destinazione viene definita tramite:

```
prefix='cleaned/single_file/'
```

indicando che il file appartiene al livello **cleaned** della pipeline e che viene salvato come dataset elaborato singolarmente.

- Contestualmente all'upload vengono associati alcuni metadati descrittivi:

```
metadata={
    'population': lst_population,
    'level': 'cleaned_01',
    'file_code': file_code,
    'source': 'ADNI'
}
```

```
}
```

In particolare:

- population contiene la lista delle coorti ADNI effettivamente presenti nel dataset;
- level identifica il livello di processamento dei dati, in questo caso cleaned_01;
- file_code rappresenta il codice univoco del dataset;
- source indica la sorgente originaria dei dati, ovvero lo studio ADNI.
- L'oggetto restituito:

result

contiene generalmente l'esito dell'operazione di upload e le informazioni necessarie per identificare il file archiviato nel Data Lake.

Attraverso questa operazione il dataset viene reso disponibile per le successive fasi della pipeline, garantendo al tempo stesso la conservazione delle informazioni necessarie per la sua gestione, riproducibilità e tracciabilità nel tempo.

In sintesi:

Il codice carica nel Data Lake il dataset finale armonizzato (final_df), salvandolo nella cartella dedicata ai dati puliti e associandovi metadati descrittivi quali popolazioni ADNI presenti, livello di processamento, codice identificativo del dataset e sorgente dei dati, garantendo così tracciabilità e corretta gestione delle versioni.